

STAR Gateway Implementation Primer

Kotlin/Spring Boot 3 Backend for Robot Control

STAR Development Team

December 2025

Contents

1 Project Overview

1.1 Purpose

The `star-gateway` is the central backend service running on the Raspberry Pi 5. It provides:

- **GraphQL API** for telemetry, settings, and navigation queries
- **gRPC service** for low-latency motor commands
- **Hardware adapters** for ESP32 communication (SPI + TCP)
- **ROS 2 bridge** for SLAM and navigation integration
- **Prometheus metrics** for monitoring

1.2 Technology Stack

Component	Technology	Version
Language	Kotlin	1.9.22
Framework	Spring Boot	3.2.x
GraphQL	Spring GraphQL	1.2.x
gRPC	grpc-spring-boot-starter	3.1.x
Hardware Access	Pi4J	2.6.x
Build Tool	Gradle (Kotlin DSL)	8.x
JVM	OpenJDK	17

Table 1: Technology Stack

1.3 Architecture Overview

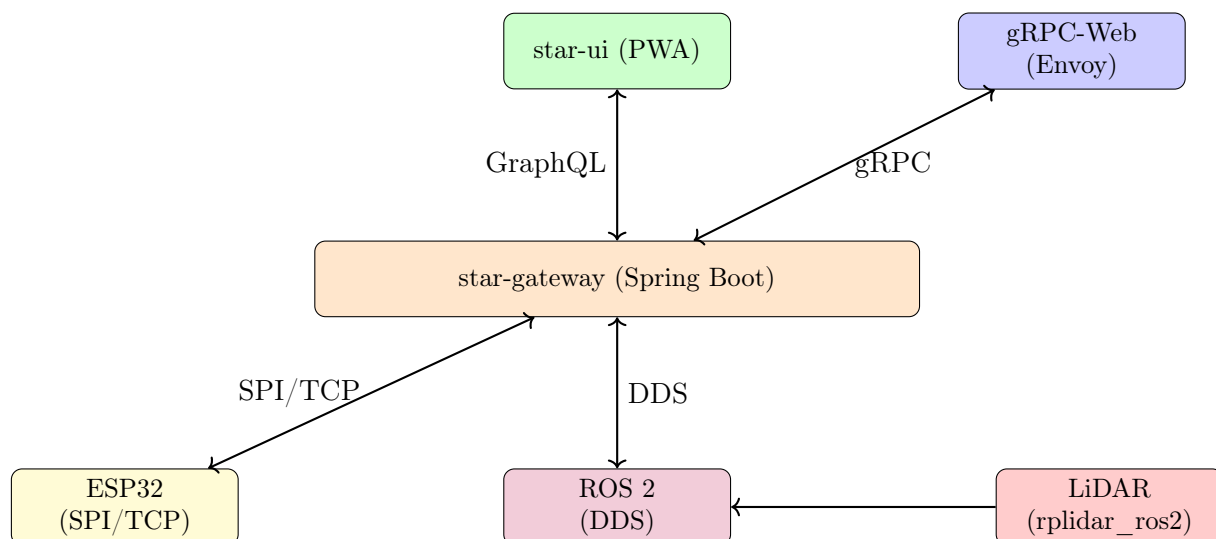


Figure 1: Gateway Architecture

2 Project Structure

```

1 star-gateway/
2 |-- build.gradle.kts           # Build configuration
3 |-- settings.gradle.kts       # Project settings
4 |-- src/
5 |   +-- main/
6 |       |-- kotlin/com/star/gateway/
7 |           |-- StarGatewayApplication.kt
8 |           |-- config/
9 |               |-- GraphQLConfig.kt
10 |               |-- GrpcConfig.kt
11 |               +-- CorsConfig.kt
12 |               |-- graphql/
13 |                   |-- TelemetryController.kt
14 |                   |-- SettingsController.kt
15 |                   +-- NavigationController.kt
16 |                   |-- grpc/
17 |                       +-- MotorControlGrpcService.kt
18 |                       |-- service/
19 |                           |-- MotorControlService.kt
20 |                           |-- TelemetryService.kt
21 |                           |-- NavigationService.kt
22 |                           +-- SensorFusionService.kt
23 |                           |-- hardware/
24 |                               |-- Esp32SpiAdapter.kt
25 |                               |-- Esp32TcpAdapter.kt
26 |                               |-- RplidarAdapter.kt
27 |                               +-- Ros2Bridge.kt
28 |                               +-- model/
29 |                                   |-- TelemetryData.kt
30 |                                   |-- MotorCommand.kt
31 |                                   +-- NavigationState.kt
32 |                               |-- proto/
33 |                                   +-- motor_control.proto
34 |                               +-- resources/
35 |                                   |-- application.yml
36 |                                   +-- graphql/
37 |                                       +-- schema.graphqls
38 +-- deployment/
39     |-- star-gateway.service     # systemd unit
40     +-- envoy.yaml               # gRPC-Web proxy config

```

3 Implementation Tasks

3.1 Phase 1: Project Setup

3.1.1 Task 1.1: Gradle Build Configuration

Create build.gradle.kts:

Listing 1: build.gradle.kts

```
1 plugins {
2     kotlin("jvm") version "1.9.22"
3     kotlin("plugin.spring") version "1.9.22"
4     id("org.springframework.boot") version "3.2.4"
5     id("io.spring.dependency-management") version "1.1.4"
6     id("com.google.protobuf") version "0.9.4"
7 }
8
9 java {
10     sourceCompatibility = JavaVersion.VERSION_17
11 }
12
13 repositories {
14     mavenCentral()
15 }
16
17 dependencies {
18     // Spring Boot Core
19     implementation("org.springframework.boot:spring-boot-starter-webflux")
20     implementation("org.springframework.boot:spring-boot-starter-actuator"
21         )
22
23     // GraphQL
24     implementation("org.springframework.boot:spring-boot-starter-graphql")
25
26     // gRPC
27     implementation("net.devh:grpc-spring-boot-starter:3.1.0.RELEASE")
28     implementation("io.grpc:grpc-kotlin-stub:1.4.1")
29     implementation("io.grpc:grpc-protobuf:1.60.1")
30     implementation("com.google.protobuf:protobuf-kotlin:3.25.2")
31
32     // Kotlin Coroutines
33     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.8.0")
34     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:1.8.0"
35         ")
36
37     // Pi4J for Hardware Access (RPi5 support)
38     implementation("com.pi4j:pi4j-core:2.6.1")
39     implementation("com.pi4j:pi4j-plugin-raspberrypi:2.6.1")
40     implementation("com.pi4j:pi4j-plugin-gpiod:2.6.1")
41     implementation("com.pi4j:pi4j-plugin-linuxfs:2.6.1")
42
43     // Metrics
44     implementation("io.micrometer:micrometer-registry-prometheus")
45
46     // JSON
47     implementation("com.fasterxml.jackson.module:jackson-module-kotlin")
48
49     // Kotlin
50     implementation("org.jetbrains.kotlin:kotlin-reflect")
51
52     // Testing
53     testImplementation("org.springframework.boot:spring-boot-starter-test"
54         )
55 }
```

```

52 }
53
54 protobuf {
55     protoc {
56         artifact = "com.google.protobuf:protoc:3.25.2"
57     }
58     plugins {
59         id("grpc") {
60             artifact = "io.grpc:protoc-gen-grpc-java:1.60.1"
61         }
62         id("grpc_kotlin") {
63             artifact = "io.grpc:protoc-gen-grpc-kotlin:1.4.1:jdk8@jar"
64         }
65     }
66     generateProtoTasks {
67         all().forEach {
68             it.plugins {
69                 id("grpc")
70                 id("grpc_kotlin")
71             }
72             it.builtins {
73                 id("kotlin")
74             }
75         }
76     }
77 }

```

3.1.2 Task 1.2: Application Configuration

Create `src/main/resources/application.yml`:

Listing 2: `application.yml`

```

1 spring:
2   application:
3     name: star-gateway
4   graphql:
5     graphiql:
6       enabled: true
7       path: /graphiql
8     websocket:
9       path: /graphql-ws
10    schema:
11      locations: classpath:graphql/
12
13 server:
14   port: 8080
15
16 grpc:
17   server:
18     port: 9090
19
20 star:
21   esp32:

```

```

22     spi:
23         enabled: true
24         bus: 0
25         chip-select: 0
26         speed-hz: 10000000
27     tcp:
28         enabled: true
29         host: 192.168.4.2
30         port: 5000
31         timeout-ms: 100
32     lidar:
33         enabled: true
34         topic: /scan
35
36 management:
37     endpoints:
38         web:
39             exposure:
40                 include: health,prometheus,info,metrics
41     endpoint:
42         health:
43             show-details: always
44
45 logging:
46     level:
47         com.star.gateway: DEBUG

```

3.2 Phase 2: GraphQL Implementation

3.2.1 Task 2.1: GraphQL Schema

Create src/main/resources/graphql/schema.graphqls:

Listing 3: schema.graphqls

```

1  type Query {
2      telemetry: Telemetry!
3      systemStatus: SystemStatus!
4      settings: Settings!
5      navigationState: NavigationState!
6      telemetryHistory(limit: Int = 100): [TelemetrySnapshot!]!
7  }
8
9  type Mutation {
10     setMode(mode: RobotMode!): Boolean!
11     emergencyStop: Boolean!
12     setSettings(input: SettingsInput!): Settings!
13     addWaypoint(x: Float!, y: Float!): Waypoint!
14     clearWaypoints: Boolean!
15     startNavigation: Boolean!
16     stopNavigation: Boolean!
17 }
18
19 type Subscription {

```

```
20     telemetryStream: Telemetry!
21     lidarScan: LidarScan!
22     eventLog: Event!
23 }
24
25 type Telemetry {
26     imu: ImuData!
27     gps: GpsData!
28     battery: Float!
29     wifiSignal: Int!
30     cpuUsage: Float!
31     temperature: Float!
32     motorLoad: Float!
33     timestamp: String!
34 }
35
36 type ImuData {
37     pitch: Float!
38     roll: Float!
39     yaw: Float!
40     velocity: Float!
41     accelerationX: Float!
42     accelerationY: Float!
43     accelerationZ: Float!
44 }
45
46 type SystemStatus {
47     connectionStatus: ConnectionStatus!
48     mode: RobotMode!
49     esp32Connected: Boolean!
50     lidarConnected: Boolean!
51     rosConnected: Boolean!
52 }
53
54 type NavigationState {
55     currentPosition: Position!
56     path: [Position!]!
57     waypoints: [Waypoint!]!
58     isNavigating: Boolean!
59 }
60
61 enum RobotMode {
62     IDLE
63     MANUAL
64     AUTONOMOUS
65     MAPPING
66     EMERGENCY_STOP
67 }
68
69 enum ConnectionStatus {
70     CONNECTED
71     DISCONNECTED
72     CONNECTING
73     ERROR
```

```

74 }
75
76 input SettingsInput {
77     maxSpeed: Float
78     safetyLockEnabled: Boolean
79     autonomousEnabled: Boolean
80 }

```

3.2.2 Task 2.2: GraphQL Controller

Create src/main/kotlin/com/star/gateway/graphql/TelemetryController.kt:

Listing 4: TelemetryController.kt

```

1 package com.star.gateway.graphql
2
3 import com.star.gateway.model.*
4 import com.star.gateway.service.TelemetryService
5 import com.star.gateway.service.MotorControlService
6 import kotlinx.coroutines.flow.Flow
7 import org.springframework.graphql.data.method.annotation.*
8 import org.springframework.stereotype.Controller
9 import reactor.core.publisher.Flux
10
11 @Controller
12 class TelemetryController(
13     private val telemetryService: TelemetryService,
14     private val motorControlService: MotorControlService
15 ) {
16     @QueryMapping
17     suspend fun telemetry(): Telemetry {
18         return telemetryService.getCurrentTelemetry()
19     }
20
21     @QueryMapping
22     suspend fun systemStatus(): SystemStatus {
23         return telemetryService.getSystemStatus()
24     }
25
26     @MutationMapping
27     suspend fun emergencyStop(): Boolean {
28         val result = motorControlService.emergencyStop()
29         return result.success
30     }
31
32     @MutationMapping
33     suspend fun setMode(@Argument mode: RobotMode): Boolean {
34         return telemetryService.setMode(mode)
35     }
36
37     @SubscriptionMapping
38     fun telemetryStream(): Flux<Telemetry> {
39         return telemetryService.telemetryFlow().asFlux()
40     }

```

41 }

3.3 Phase 3: gRPC Implementation

3.3.1 Task 3.1: Protobuf Definition

Create src/main/proto/motor_control.proto:

Listing 5: motor_control.proto

```

1 syntax = "proto3";
2
3 package star.motor;
4
5 option java_package = "com.star.gateway.proto";
6 option java_multiple_files = true;
7
8 service MotorControl {
9     rpc SetVelocity(VelocityCommand) returns (CommandResponse);
10    rpc EmergencyStop(EmptyRequest) returns (CommandResponse);
11    rpc StreamEncoderData(EmptyRequest) returns (stream EncoderData);
12 }
13
14 message VelocityCommand {
15     float left_velocity = 1;
16     float right_velocity = 2;
17     uint64 timestamp = 3;
18 }
19
20 message CommandResponse {
21     bool success = 1;
22     string message = 2;
23     uint64 latency_us = 3;
24 }
25
26 message EncoderData {
27     int32 motor_id = 1;
28     int64 ticks = 2;
29     float velocity = 3;
30     uint64 timestamp = 4;
31 }
32
33 message EmptyRequest {}

```

3.3.2 Task 3.2: gRPC Service

Create src/main/kotlin/com/star/gateway/grpc/MotorControlGrpcService.kt:

Listing 6: MotorControlGrpcService.kt

```

1 package com.star.gateway.grpc
2
3 import com.star.gateway.proto.*
4 import com.star.gateway.service.MotorControlService

```

```

5 import kotlinx.coroutines.flow.Flow
6 import kotlinx.coroutines.flow.flow
7 import net.devh.boot.grpc.server.service.GrpcService
8 import org.slf4j.LoggerFactory
9
10 @GrpcService
11 class MotorControlGrpcService(
12     private val motorControlService: MotorControlService
13 ) : MotorControlGrpcKt.MotorControlCoroutineImplBase() {
14
15     private val logger = LoggerFactory.getLogger(javaClass)
16
17     override suspend fun setVelocity(request: VelocityCommand):
18         CommandResponse {
19         val startTime = System.nanoTime()
20
21         logger.debug("gRPC SetVelocity: left=${request.leftVelocity},
22             right=${request.rightVelocity}")
23
24         val result = motorControlService.setVelocity(
25             left = request.leftVelocity,
26             right = request.rightVelocity
27         )
28
29         val latencyUs = (System.nanoTime() - startTime) / 1000
30
31         return CommandResponse.newBuilder()
32             .setSuccess(result.success)
33             .setMessage(result.message)
34             .setLatencyUs(latencyUs)
35             .build()
36     }
37
38     override suspend fun emergencyStop(request: EmptyRequest):
39         CommandResponse {
40         logger.warn("gRPC EmergencyStop received")
41         val result = motorControlService.emergencyStop()
42
43         return CommandResponse.newBuilder()
44             .setSuccess(result.success)
45             .setMessage(result.message)
46             .build()
47     }
48
49     override fun streamEncoderData(request: EmptyRequest): Flow<
50         EncoderData> = flow {
51         while (true) {
52             val encoders = motorControlService.getEncoderData()
53             encoders.forEach { (motorId, data) ->
54                 emit(
55                     EncoderData.newBuilder()
56                         .setMotorId(motorId)
57                         .setTicks(data.ticks)
58                         .setVelocity(data.velocity)

```

```

55         .setTimestamp(System.currentTimeMillis())
56         .build()
57     )
58 }
59     kotlinx.coroutines.delay(10) // 100Hz
60 }
61 }
62 }

```

3.4 Phase 4: Hardware Adapters

3.4.1 Task 4.1: ESP32 SPI Adapter

Create `src/main/kotlin/com/star/gateway/hardware/Esp32SpiAdapter.kt`:

Listing 7: Esp32SpiAdapter.kt

```

1  package com.star.gateway.hardware
2
3  import com.pi4j.Pi4J
4  import com.pi4j.context.Context
5  import com.pi4j.io.spi.Spi
6  import com.pi4j.io.spi.SpiMode
7  import com.star.gateway.model.CommandResult
8  import org.springframework.beans.factory.annotation.Value
9  import org.springframework.stereotype.Component
10 import java.nio.ByteBuffer
11 import java.nio.ByteOrder
12 import javax.annotation.PostConstruct
13 import javax.annotation.PreDestroy
14
15 @Component
16 class Esp32SpiAdapter(
17     @Value("\${star.esp32.spi.enabled:true}")
18     private val enabled: Boolean,
19     @Value("\${star.esp32.spi.bus:0}")
20     private val spiBus: Int,
21     @Value("\${star.esp32.spi.chip-select:0}")
22     private val chipSelect: Int,
23     @Value("\${star.esp32.spi.speed-hz:1000000}")
24     private val speedHz: Int
25 ) {
26     companion object {
27         const val CMD_SET_VELOCITY: Byte = 0x01
28         const val CMD_EMERGENCY_STOP: Byte = 0x02
29         const val CMD_GET_ENCODERS: Byte = 0x03
30         const val RESPONSE_OK: Byte = 0x00
31     }
32
33     private var pi4j: Context? = null
34     private var spi: Spi? = null
35     private var connected = false
36
37     @PostConstruct

```

```

38 fun initialize() {
39     if (!enabled) return
40
41     try {
42         pi4j = Pi4J.newAutoContext()
43         val spiConfig = Spi.newConfigBuilder(pi4j)
44             .id("esp32-spi")
45             .bus(spiBus)
46             .chipSelect(chipSelect)
47             .baud(speedHz)
48             .mode(SpiMode.MODE_0)
49             .build()
50
51         spi = pi4j?.create(spiConfig)
52         connected = testConnection()
53     } catch (e: Exception) {
54         connected = false
55     }
56 }
57
58 @PreDestroy
59 fun shutdown() {
60     spi?.close()
61     pi4j?.shutdown()
62 }
63
64 suspend fun sendVelocityCommand(left: Float, right: Float):
65     CommandResult {
66     if (!enabled || spi == null) {
67         throw IllegalStateException("SPI not initialized")
68     }
69
70     val buffer = ByteBuffer.allocate(9)
71         .order(ByteOrder.LITTLE_ENDIAN)
72         .put(CMD_SET_VELOCITY)
73         .putFloat(left)
74         .putFloat(right)
75
76     val txData = buffer.array()
77     val rxData = ByteArray(2)
78
79     synchronized(this) {
80         spi!!.transfer(txData, rxData)
81     }
82
83     return if (rxData[0] == RESPONSE_OK) {
84         CommandResult.success("Velocity set: left=$left, right=$right"
85             )
86     } else {
87         CommandResult.failure("ESP32 error: ${rxData[0]}")
88     }
89 }
90
91 suspend fun emergencyStop(): CommandResult {

```

```

90     val txData = byteArrayOf(CMD_EMERGENCY_STOP)
91     val rxData = ByteArray(2)
92
93     synchronized(this) {
94         spi?.transfer(txData, rxData)
95     }
96
97     return if (rxData[0] == RESPONSE_OK) {
98         CommandResult.success("Emergency stop executed")
99     } else {
100         CommandResult.failure("Emergency stop failed")
101     }
102 }
103
104 private fun testConnection(): Boolean {
105     // Ping command implementation
106     return true
107 }
108
109 fun isConnected(): Boolean = connected && enabled
110 }

```

3.4.2 Task 4.2: ESP32 TCP Adapter

Create `src/main/kotlin/com/star/gateway/hardware/Esp32TcpAdapter.kt` as fallback communication channel using JSON over TCP socket.

3.4.3 Task 4.3: ROS 2 Bridge

Create `src/main/kotlin/com/star/gateway/hardware/Ros2Bridge.kt` to interface with ROS 2 via DDS or `rosbridge`.

3.5 Phase 5: Deployment

3.5.1 Task 5.1: systemd Service

Create `deployment/star-gateway.service`:

Listing 8: `star-gateway.service`

```

1  [Unit]
2  Description=STAR Robot Gateway
3  After=network-online.target
4  Wants=network-online.target
5
6  [Service]
7  Type=simple
8  User=root
9  WorkingDirectory=/opt/star-gateway
10 ExecStart=/usr/bin/java -Xmx512m -jar star-gateway.jar
11 Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
12 Restart=always
13 RestartSec=10
14

```

```

15 [Install]
16 WantedBy=multi-user.target

```

3.5.2 Task 5.2: Envoy Proxy (gRPC-Web)

Create deployment/envoy.yaml for browser gRPC support:

Listing 9: envoy.yaml

```

1 static_resources:
2   listeners:
3   - name: listener_0
4     address:
5       socket_address: { address: 0.0.0.0, port_value: 8090 }
6     filter_chains:
7     - filters:
8       - name: envoy.filters.network.http_connection_manager
9         typed_config:
10            "@type": type.googleapis.com/envoy.extensions.filters.network.
11              http_connection_manager.v3.HttpConnectionManager
12          codec_type: auto
13          stat_prefix: ingress_http
14          route_config:
15            name: local_route
16            virtual_hosts:
17            - name: local_service
18              domains: ["*"]
19              routes:
20              - match: { prefix: "/" }
21                route:
22                  cluster: grpc_service
23                  timeout: 0s
24                  max_stream_duration:
25                    grpc_timeout_header_max: 0s
26              cors:
27                allow_origin_string_match:
28                - prefix: "*"
29                allow_methods: GET, PUT, DELETE, POST, OPTIONS
30                allow_headers: keep-alive,user-agent,cache-control,content
31                  -type,content-transfer-encoding,x-accept-content-
32                  transfer-encoding,x-accept-response-streaming,x-user-
33                  agent,x-grpc-web,grpc-timeout
34                expose_headers: grpc-status,grpc-message
35          http_filters:
36          - name: envoy.filters.http.grpc_web
37          - name: envoy.filters.http.cors
38          - name: envoy.filters.http.router
39
40 clusters:
41 - name: grpc_service
42   connect_timeout: 0.25s
43   type: logical_dns
44   http2_protocol_options: {}
45   lb_policy: round_robin

```

```

42     load_assignment:
43       cluster_name: cluster_0
44       endpoints:
45         - lb_endpoints:
46           - endpoint:
47               address:
48                 socket_address:
49                   address: localhost
50                   port_value: 9090

```

4 Build and Run

4.1 Local Development

```

1  cd star-gateway
2
3  # Build
4  ./gradlew build
5
6  # Run locally (without Pi4J hardware)
7  ./gradlew bootRun --args='--star.esp32.spi.enabled=false'
8
9  # Build JAR for deployment
10 ./gradlew bootJar
11
12 # Output: build/libs/star-gateway-*.jar

```

4.2 Deploy to Pi5

```

1  # Copy JAR to Pi5
2  scp build/libs/star-gateway-*.jar root@star-robot.local:/opt/star-gateway/
   star-gateway.jar
3
4  # SSH and restart service
5  ssh root@star-robot.local
6  systemctl restart star-gateway
7  journalctl -u star-gateway -f

```

4.3 API Endpoints

Endpoint	Protocol	Purpose
http://star-robot.local:8080/graphql	GraphQL	Queries, Mutations
ws://star-robot.local:8080/graphql-ws	WebSocket	Subscriptions
http://star-robot.local:8080/graphiql	HTTP	GraphQL IDE
star-robot.local:9090	gRPC	Motor commands
star-robot.local:8090	gRPC-Web	Browser gRPC
http://star-robot.local:8080/actuator/*	HTTP	Health, Metrics

Table 2: API Endpoints

5 Testing

5.1 GraphQL Testing

Use GraphQL at `http://star-robot.local:8080/graphql`:

Listing 10: Sample GraphQL Queries

```
1  # Get current telemetry
2  query {
3    telemetry {
4      battery
5      temperature
6      imu { pitch roll yaw }
7    }
8  }
9
10 # Emergency stop
11 mutation {
12   emergencyStop
13 }
14
15 # Subscribe to telemetry stream
16 subscription {
17   telemetryStream {
18     battery
19     timestamp
20   }
21 }
```

5.2 gRPC Testing

Use `grpcurl`:

```
1  # List services
2  grpcurl -plaintext localhost:9090 list
3
4  # Set velocity
5  grpcurl -plaintext -d '{"leftVelocity": 0.5, "rightVelocity": 0.5}' \
6    localhost:9090 star.motor.MotorControl/SetVelocity
7
8  # Emergency stop
9  grpcurl -plaintext -d '{} ' \
10   localhost:9090 star.motor.MotorControl/EmergencyStop
```

6 References

- Spring GraphQL: <https://docs.spring.io/spring-graphql/reference/>
- grpc-spring-boot-starter: <https://yidongnan.github.io/grpc-spring-boot-starter/>
- Pi4J v2: <https://pi4j.com/>

- Envoy gRPC-Web: https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/grpc_web_filter
- STAR Software Architecture Primer: Section 10